Journal of Cloud Computing:
Advances, Systems and Applications

RESEARCH

Open Access

A novel virtual machine placement algorithm based on grey wolf optimization

Hao Feng[1], Haoyu Li[1], Yuming Liu[1*], Kun Cao[1] and Xiumin Zhou[1]

Abstract

Virtual machine placement is a typical NP-hard problem in the field of cloud computing. Unreasonable placement schemes can result in low resource utilization and high energy consumption. To address these issues, this paper proposes a heuristic algorithm named Adaptive Boundary Grey Wolf Optimization (ABGWO). The objective is to minimize the number of physical servers, and the evaluation is conducted using Microsoft's Azure Trace dataset from 2020. Comparative experiments are performed with genetic algorithm, particle swarm optimization algorithm, artificial bee colony algorithm, moth search algorithm, and differential evolution algorithms. The experimental results demonstrate that the ABGWO algorithm outperforms other algorithms in terms of solution quality and stability.

Keywords Virtual machine placement, Grey wolf optimization (GWO), Adaptive boundary, Cloud computing

Introduction

In response to the increasing demand for cloud computing services, major players like Microsoft, Google, and Amazon have embraced cloud service models, revolutionizing modern data center operations. Leveraging virtualization technology, these data centers abstract physical resources such as CPU, memory, and storage into virtual machines (VMs) of varying sizes. This allows for the simultaneous deployment of multiple VMs on a single physical machine (PM), enhancing resource utilization and reducing maintenance costs while meeting stringent service quality requirements.

However, despite these advancements, global cloud service providers are confronted with the challenge of excessive energy consumption within their data centers [1]. Most cloud servers run at low utilization rates [1], and the average CPU utilization of all running PMs in cloud data centers is between 15% and 20% [2]. Many PMs are severely underutilized, with idle machines accounting for up to 70% of peak energy consumption [3]. Unreasonable VM placement schemes can lead to high energy consumption and low resource utilization problems in cloud data centers. Therefore, optimizing the placement scheme of virtual machines can reduce energy consumption and resource waste [4–8]. The virtual machine placement problem(VMPP) is a typical multidimensional binning problem (MBPP) [9], where physical machines (PMs) act as boxes and virtual machines are the items to be placed in these boxes. Each item has three different dimensions: CPU, memory, and disk capacity. Since the multidimensional bin packing problem is a typical NP-hard problem [10–12], there is no solution with polynomial time complexity unless P=NP [13]. Therefore, only approximate solutions can be obtained for small-scale virtual machine placement problems.

Although heuristic algorithms are often feasible solutions, their effectiveness is limited when the problem size increases [14], while swarm intelligence algorithms are expected to achieve better results. The GWO algorithm, developed by Mirjalili et al. [15], has rapidly become one of the most prominent swarm intelligence algorithms, inspired by the cooperative hunting strategies observed in grey wolf packs [16]. GWO has gained significant interest from researchers in various scientific and engineering domains, and widely applied in fields including

*Correspondence:
Yuming Liu
19031105004@mails.guet.edu.cn
[1] School of Computer Science and Information Security, Guilin University of Electronic Technology, Guilin 541004, Guangxi, China

engineering, machine learning, medical and bioinformatics, networking, environmental sciences, and image processing [17]

The main contributions of this paper are as follows:

1. A model for the VMPP in cloud computing is developed, incorporating various constraints to better address real-world challenges.

2. An Adaptive Boundary Grey Wolf Optimization (ABGWO) algorithm is proposed to solve the VMPP by minimizing the number of servers required for placement.

3. Comprehensive experiments on real-world datasets are conducted, demonstrating that the ABGWO algorithm outperforms other algorithms in terms of solution quality, stability, convergence speed, and running time.

The paper is structured as follows: "Related work" section provides an overview of related work. In "Problem statement" section, the definition of VMPP is presented. "Grey wolf optimization" section introduces the GWO algorithm. The proposed ABGWO algorithm is detailed in "Adaptive boundary grey wolf optimization" section, followed by the presentation of experimental results and discussion in "Experimental evaluation" section. Finally, "Conclusion" section offers concluding remarks along with the future scope of the work.

Related work

Virtualization technology is an essential component of cloud computing, enabling resource virtualization to improve resource utilization. However, as the number of virtual machines in cloud environments increases, low utilization of hardware and software resources and high energy consumption have become common issues. An effective virtual machine placement strategy can fully leverage available hardware resources from the outset, preventing resource wastage, thereby improving resource utilization and reducing energy consumption. In recent years, research on VMPP has primarily focused on load balancing, resource optimization, and energy efficiency. These challenges are often addressed using heuristic algorithms, metaheuristic approaches, machine learning algorithms, and statistical methods.

Jyoti, A et al. [18] take dynamic resource allocation as the central goal and adopted a new method based on load balancing and service brokerage to address the dynamic provisioning of resources in cloud computing environments. Belgacem, A et al. [19] introduce a dynamic resource allocation model, and a multi-objective search algorithm named Spacing Multi-Objective Antlion algorithm (S-MOAL), aiming to minimize both makespan and the cost associated with using VMs. In order to enhance the task scheduling performance in cloud task allocation and reduce the total task completion time, Lavanya, M et al. [20] propose two scheduling algorithms: TBTS (Threshold-based Task Scheduling Algorithm) and SLA-LB (Service Level Agreement-based Load Balancing) algorithm. Liu, B et al. [21] present an enhanced container scheduling algorithm tailored for big data applications known as Kubernetes-based Particle Swarm Optimization (K-PSO), which convergence is reduced by approximately half the time compared to the basic PSO algorithm. Kumar, KRP et al. [22] introduce the Crow Search Algorithm (CSA) for cloud task scheduling, which showed superior performance in minimizing completion time and finding virtual machines suitable for tasks, M et al. [23] modify the transfer function of Binary Particle Swarm Optimization (BPSO) to fairly allocate applications to virtual machines and optimize the Quality of Service (QoS) parameters to meet the needs of end-users. Arani, M et al. [24] proposed a VMP strategy based on the Best Fit Decreasing (BFD) algorithm with the goal of reducing energy consumption. Compared to other reference algorithms, this strategy significantly reduces energy consumption and SLA violations. A method called "Space-Aware Best Fit Decreasing" (SABFD) was proposed by Wang et al. [25], which uses a virtual machine selection strategy known as "High CPU Utilization-Based" (HS). Experimental results show that this method is effective in reducing energy consumption in data centers. Abohamama, AS et al. [9] propose a hybrid VM Placement (VMP) algorithm based on an improved permutation-based genetic algorithm and a multidimensional resource-aware best-fit allocation strategy. Rashida, SY et al. [26] introduce a correlation-aware VMP algorithm named MGGAVP, based on the hybridization of Grouping Genetic Algorithm and Hill-Climbing, extended for multi-cloud environments to minimize energy costs. Azizi, S et al. [27] devise a Greedy Randomized VMP (GRVMP) algorithm to solve the problems of heterogeneity of VM and PM, multi-dimensionality of resources, and scale expansion of cloud data centers (CDCs). Duong-Ba, T et al. [28] propose Multi-level Join VM Placement and Migration (MJPM) algorithms, which uses a relaxed convex optimization framework to approximately solve the optimal solution of VMP and migration in the data center. Gharehpasha, S et al. [5] combine discrete multi-objective and chaotic functions, employing the Sine-Cosine Algorithm and Salp Swarm Algorithm to solve the VMPP. Tabrizchi, H et al. utilize the ant colony algorithm [29], Baalamurugan, KM et al. [30] utilize the multi-objective whale swarm technique for VMP, and Caviglione, L et al. [31]

leverage deep reinforcement learning for multi-objective VMP in cloud datacenters. Azizi, S et al. [32] propose the MinPR algorithm, which focuses on power consumption and resource wastage optimization for VMP. Tseng, FH et al. [33] introduce the Service-Oriented PM Selection (SOPMS) algorithm and the Link-Aware VMP (LAVMP) algorithm. Finally, Tripathi, A et al. [34] apply a modified dragonfly algorithm to enhance resource utilization in cloud data centers by optimizing VMP. Table 1 provides a summary of existing work, which detailing the pros and cons of each approach.

For VMPP, existing methods rarely take into account convergence speed, solution quality, solution stability and time consumption at the same time. In this paper, our work considers all four of these metrics simultaneously. And our method achieves the best results at the comprehensive level of these four indicators at the same time compared to the comparison algorithm.

Problem statement

Figure 1 shows the resource scheduling in cloud computing centers. Initially, the user initiates a VM creation request as shown in step 1. Then, as shown in steps 2, 3, and 4, the VM creation request is transmitted to the distributed dispatch center through the network link. Afterwards, the VM creation request handler places the request into a pool designated for VM creation requests, as shown in step 5. Finally, the scheduler generates and executes the ultimate placement scheme based on the VM creation requests within the pool, as depicted in

Table 1 Summary of existing work

Approach	Pros	Cons
Jyoti, A et al. [18]	Effective load balancing and dynamic provisioning	Limited to specific scenarios of load balancing
Belgacem, A et al. [19]	Minimizes both makespan and VM cost	High computational complexity
Lavanya, M et al. [20]	Enhances task scheduling, reduces completion time	Limited scalability with larger task pools
Liu, B et al. [21]	Reduced convergence time for big data applications	Not generalized beyond big data context
Kumar, KRP et al. [22]	Improves VM-task matching, minimizes task completion time	Limited focus on other VMP performance aspects
M et al. [23]	Optimizes QoS, fair resource allocation	Requires specific modifications for different scenarios
Arani, M et al. [24]	Reduces energy use and SLA violations	Limited adaptability to nonenergy-focused tasks
Wang et al. [25]	Effective in energy consumption reduction	High CPU utilization can reduce VM diversity
Abohamama, AS et al. [9]	Multidimensional resource allocation	High complexity in genetic permutation calculations
Rashida, SY et al. [26]	Energy-efficient, correlation-aware for multi-cloud	Limited to environments with strong correlations
Azizi, S et al. (GRVMP) [27]	Handles VM/PM heterogeneity and scaling	Complexity grows with increased multi-dimensionality
Duong-Ba, T et al. [28]	Optimizes VMP and migration with relaxed convex approach	Approximation may lead to suboptimal results
Gharehpasha, S et al. [5]	Integrates chaotic functions for stability	Chaotic functions may reduce predictability
Tabrizchi, H et al. [29]	Utilizes ant colony optimization effectively	High resource needs for large data centers
Baalamurugan, KM et al. [30]	Effective for multi-objective VMP	Whale swarm technique may be slow in convergence
Caviglione, L et al. [31]	Deep learning enhances multi-objective optimization	Requires high computation, not energy-efficient

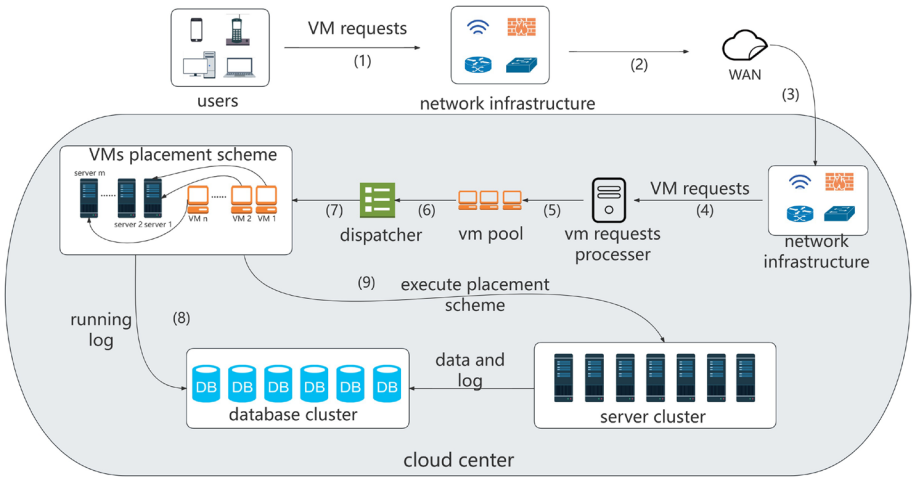


Fig. 1 A resource scheduling scenario in cloud center

steps 6, 7, 8, and 9. In the VM placement scheme, let n denote the number of VMs, with $\boldsymbol{N} = \{1, ..., n\}$ representing the set of VMs. Let m denote the number of available servers, with $\boldsymbol{S} = \{1, ..., m\}$ representing the set of servers. Assuming m is not less than the optimal solution (e.g., $m = n$). Furthermore, a VM can be described as a triple (c_j, m_j, d_j), where $j \in \boldsymbol{N}$ and c_j, m_j, d_j respectively denote the CPU quantity, memory size, and disk size of the VM. Servers, acting as containers for VMs, have three capacity indicators: CPU quantity, memory size, and disk size, denoted by C_i , M_i , and D_i, where $i \in \boldsymbol{S}$. For the sake of dimensional consistency without loss of generality, this paper assumes $C_i = M_i = D_i = 1$, and for any $j \in \boldsymbol{N}$, $c_j, m_j, d_j \leq 1$. The problem to be solved is deploying these n VMs onto servers, aiming to minimize the number of utilized servers (up to m at most).

The optimization goal and the associated constraints for this problem can be formulated as the following equations:

$$min \sum_{i \in S} y_i \tag{1}$$

s.t.

$$\sum_{i \in S} x_{ij} = 1, j \in N \tag{2}$$

$$\sum_{j \in N} x_{ij} \cdot c_j \leq C_i, i \in S \tag{3}$$

$$\sum_{j \in N} x_{ij} \cdot m_j \leq M_i, i \in S \tag{4}$$

$$\sum_{j \in N} x_{ij} \cdot d_j \leq D_i, i \in S \tag{5}$$

$$x_{ij} \in \{0, 1\}, i \in S, j \in N \tag{6}$$

$$y_i \in \{0, 1\}, i \in S \tag{7}$$

where y_i indicates whether the server i is used, $y_i = 1$ if and only if server i is used, otherwise $y_i = 0$. x_{ij} means deploying VM j to server i, $x_{ij} = 1$ if and only if VM j is deployed to server i, otherwise $x_{ij} = 0$. Equation (1) represents the minimum number of servers required to deploy n VMs. Equation (2) represents that each VM must and can only be deployed to one server. Equations (3), (4), and (5) limit the total cpu, total memory, and total disk size of VMs deployed to a particular server to not exceed the capacity of that server.

VMPP, which has been discussed in "Introduction" section, is an NP-hard problem. Therefore, we designed the ABGWO algorithm to solve it. To evaluate the quality of the solution, we considered the number of required virtual machines, including the best-case, worst-case, and mean-case. Additionally, we considered the convergence speed of the algorithms at the same scale, the solution stability using the variance (Std) of the experimental results, and the time efficiency through the experimental runtime.

Grey wolf optimization

The GWO algorithm is inspired by the leadership hierarchy of grey wolves [15]. Grey wolves are at the top of the food chain. There are four types of grey wolves

within the leadership hierarchy. These are alpha (α), beta (β), delta (δ) and omega (ω) wolves. In the GWO algorithm, alpha wolves represent the solution with the best result. Beta and delta wolves represent the second and third best solutions in the population. Omega wolves are the best solution candidates. The GWO algorithm assumes that hunting is performed by alpha, beta and delta wolves, while omega wolves follow these wolves. Grey wolves' hunting includes the following three main parts: encircling, hunting and attacking the prey.

Mathematical model in encircling the prey

In hunting, wolves encircle the prey. The formula for updating their position is as follows:

$$\overrightarrow{D} = |\overrightarrow{C} \cdot \overrightarrow{X_p}(t) - \overrightarrow{X}(t)| \tag{8}$$

$$\overrightarrow{X}(t+1) = \overrightarrow{X_p}(t) - \overrightarrow{A} \cdot \overrightarrow{D} \tag{9}$$

t represents the current iteration; $\overrightarrow{A}$, $\overrightarrow{D}$ represents the coefficient vectors; represents the position vector of the grey wolf, and $\overrightarrow{X_p}$ is the position vector of the prey. $\overrightarrow{A}$, $\overrightarrow{C}$ and $\overrightarrow{a}$ are calculated by the following formulas:

$$\overrightarrow{A} = 2\overrightarrow{a} \cdot \overrightarrow{r_1} - \overrightarrow{a} \tag{10}$$

$$\overrightarrow{C} = 2 \cdot \overrightarrow{r_2} \tag{11}$$

where $\overrightarrow{a}$ is linearly decreased from 2 to 0 over the courses of iteration. It is used to get closer to the solution range. $\overrightarrow{r_1}$ and $\overrightarrow{r_2}$ are the random vectors in range of [0,1].

A grey wolf in the position of (X, Y) can update its position according to the position of the prey (X^*, Y^*). Different places around the best agent can be reached with respect to the current position by adjusting the value of $\overrightarrow{A}$ and $\overrightarrow{C}$ vectors. A grey wolf can also update its position inside the space around the prey in any random location by using Eqs. (8) and (9).

Mathematical model for hunting mechanism

Grey wolves are led by alpha wolves to detect the location of the prey. Sometimes beta and delta wolves help the alpha wolf. GWO algorithm assumes that the best solution is the alpha wolf, and the second best and the third solutions are beta and delta wolves, respectively. For this reason, the other wolves in the population move according to the position of these three wolves. This is formulated mathematically in the following equations:

$$\begin{aligned} \overrightarrow{D_\alpha} &= |\overrightarrow{C_1} \cdot \overrightarrow{X_\alpha} - \overrightarrow{X}| \\ \overrightarrow{D_\beta} &= |\overrightarrow{C_2} \cdot \overrightarrow{X_\beta} - \overrightarrow{X}| \\ \overrightarrow{D_\delta} &= |\overrightarrow{C_3} \cdot \overrightarrow{X_\delta} - \overrightarrow{X}| \end{aligned} \tag{12}$$

and

$$\begin{aligned} \overrightarrow{X_1} &= \overrightarrow{X_\alpha} - \cdot\overrightarrow{A_1} \cdot \overrightarrow{D_\alpha} \\ \overrightarrow{X_2} &= \overrightarrow{X_\beta} - \cdot\overrightarrow{A_2} \cdot \overrightarrow{D_\beta} \\ \overrightarrow{X_3} &= \overrightarrow{X_\delta} - \cdot\overrightarrow{A_3} \cdot \overrightarrow{D_\delta} \end{aligned} \tag{13}$$

then

$$\overrightarrow{X}(t+1) = \frac{\overrightarrow{X_1}(t) + \overrightarrow{X_2}(t) + \overrightarrow{X_3}(t)}{3} \tag{14}$$

Where X_α , X_β and X_δ represent the best three wolves in each iteration, respectively. The new position of the prey is expressed as $\overrightarrow{X}(t+1)$ as the mean of the positions of three best wolves in the population.

Attacking prey

It is worth mentioning that here exploration and exploitation are important in meta-heuristic algorithms. GWO tries to trade-off between these two phases. Grey wolves finish the hunting by attacking the prey. For attacking, they must get close enough to the prey. In GWO, $\overrightarrow{a}$ value in each iteration decreased from 2 to 0. Actually, $\overrightarrow{A}$ value is also decreased by $\overrightarrow{a}$. The value $\overrightarrow{A}$ is important to a grey wolf. When $|A| < 1$, force the wolves to attack the prey. Otherwise, $|A| > 1$, wolves try to search other prey. That proves the exploration and exploitation concepts. $\overrightarrow{C}$ Parameter provides random values in each iteration. Authors emphasize that this value has an effect in exploration all time, even in the last iteration. The flowchart of GWO algorithm is shown in Fig. 2.

Adaptive boundary grey wolf optimization

While the GWO demonstrates outstanding performance across various scenarios, it encounters challenges when dealing with high-dimensional problems or those featuring a wide feature space. To address these limitations, we propose the ABGWO. ABGWO enhances the algorithm's precision by dynamically adapting the search boundaries throughout the search process,this approach enhances its performance in solving multimodal function optimization problems.

For the VMPP, two encoding schemes are utilized, namely binary coding and decimal coding, as illustrated in Figs. 3 and 4. However, for the VMPP addressed in this paper, decimal encoding proves to be significantly more

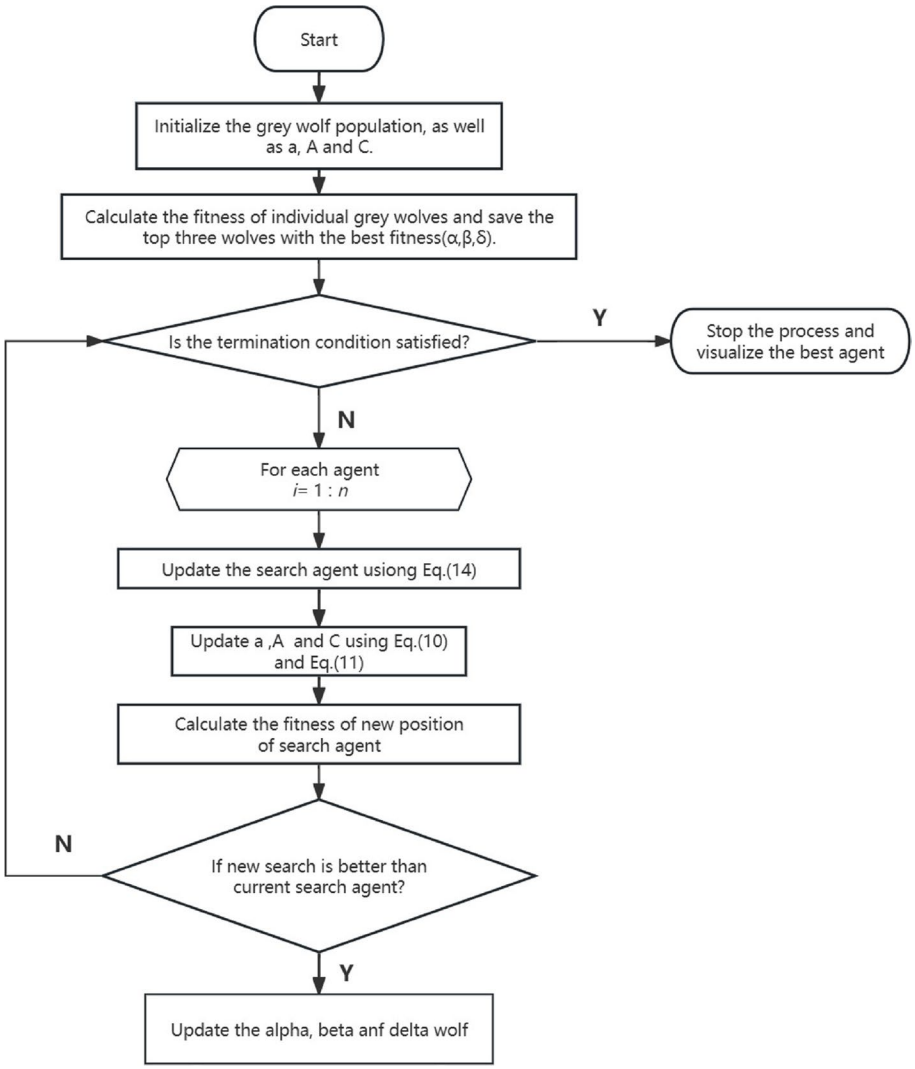


Fig. 2 The flow chart of GWO

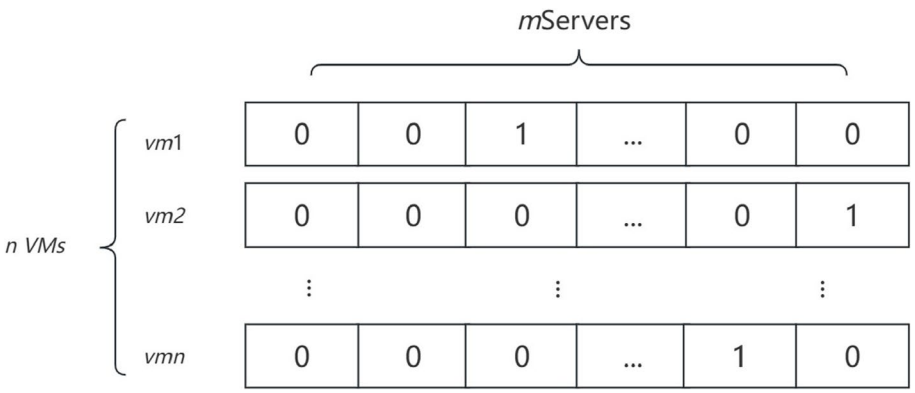


Fig. 3 The binary coding scheme of individual xi for the VMPP

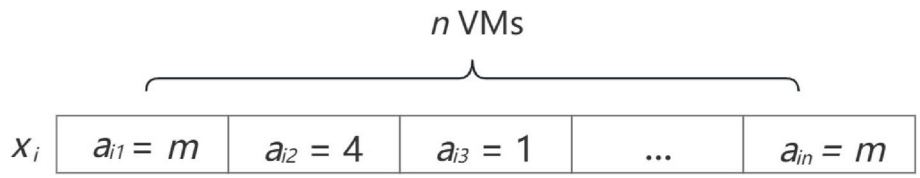


Fig. 4 The decimal coding scheme of individual xi for the VMPP

advantageous than binary encoding. Specifically, under binary encoding, the search space of VMPP expands to $(2^m)^n$, whereas under decimal encoding, the search space is notably reduced to m^n. Consequently, the decimal encoding scheme was adopted for this paper.

We define each individual of population $\boldsymbol{X}$ as $x_i = (a_{i1}, a_{i2}, ..., a_{in})$, x_i represents a possible VM deployment scheme, a_{ij} means: deploying VM j on server a_{ij}, $1 \le i \le p_{size}$, $1 \le j \le n$, $1 \le a_{ij} \le m$. Obviously the minimum value of a_{ij} is 1 and the maximum value is m. For the VM deployment scheme x_i, *num* is used to represent the number of different values of the discrete value a_{ij}. *num* means that *num* servers are needed to deploy n VMs. Obviously, the smaller the *num* the higher the quality of the solution, where p_{size} represents the number of individuals in population $\boldsymbol{X}$, n represents the number of VMs, and m represents the number of servers.

Due to the exponential time complexity of the algorithm described by the objective function and constraints in "Problem statement" section, we opt for utilizing a population intelligence optimization algorithm to address the VM placement problem, rather than employing integer programming.

Every population-based intelligent optimization algorithm incorporates elements of stochastic algorithms, inevitably introducing randomness to the solutions generated. Our proposed ABGWO algorithm is no exception to this rule. Due to the influence of randomness, the solution x_i generated by the ABGWO algorithm is often invalid, meaning that deploying virtual machines according to the scheme x_i may result in some servers having insufficient CPU, memory, or disk resources remaining. A negative value indicates that the VMs deployed on these servers have exceeded the maximum available resources.

There are typically three approaches to handling illegal solutions. The most straightforward method is to directly discard the illegal solution, yet this may lead to a decrease in the population size and a loss of diversity. Alternatively, one can attempt to rectify the illegal solution to render it legal, although this proves challenging in our problem due to the numerous constraints outlined in "Problem statement" section. Each modification of the value of a_{ij} must consider these constraints, presenting a formidable NP-hard problem. The final approach to addressing illegal solutions involves imposing penalties, thereby reducing the priority of illegal solutions within the population. These penalties should reflect the severity of the various illegal solutions.

In VMPP, the penalty function method is employed to handle illegal solutions. In order to distinguish between various illegal solutions based on their quality differences, we utilize the Augmented Lagrangian method (ALM) introduced by Bahreininejad et al. [35]. ALM mitigates the risk of ill-conditioning that may occur in the penalty method by incorporating explicit estimates of Lagrange multipliers into the objective function, resulting in what is known as the augmented Lagrange function. The ALM formulation can be represented as follows:

$$F(x) = f(x) + \mu \cdot \sum_{t=1}^{n} g_t^2(x) - \sum_{t=1}^{n} \lambda \cdot g_t(x) \tag{15}$$

where λ is the Lagrange multiplier. Unlike the usual penalty function, the main advantage of ALM is that it is not necessary to make $\mu \to \infty$; instead, the penalty coefficient μ can be kept at a small value to avoid ill conditioning due to the presence of the Lagrange multiplier λ. $f(x)$ is the original evaluation function, and $F(x)$ is the evaluation function with the addition of the augmented Lagrange penalty term. For the legal solution, $\mu \cdot \sum_{t=1}^{n} g_t^2(x) - \sum_{t=1}^{n} \lambda \cdot g_t(x)$ is always zero, while for the illegal solution, $\mu \cdot \sum_{t=1}^{n} g_t^2(x) - \sum_{t=1}^{n} \lambda \cdot g_t(x)$ is the imposed penalty value.

By incorporating the ALM penalty term into the evaluation function $F(x)$, we effectively transform the objective function with constraints outlined in "Problem statement" section into an unconstrained problem. Let $y_i = F(x_i)$, where y_i represents the output of individual x_i processed by the evaluation function $F(x)$, denoting the number of servers required for deploying VMs according to the scheme x_i. Thus, the population X is processed by the function $F(X)$ to derive Y. Let $y_{best} = min(Y)$, where y_{best} represents the minimum value in Y, indicating the best deployment scheme within the current population. If $y_{best} < m$, implying that n VMs can be deployed using

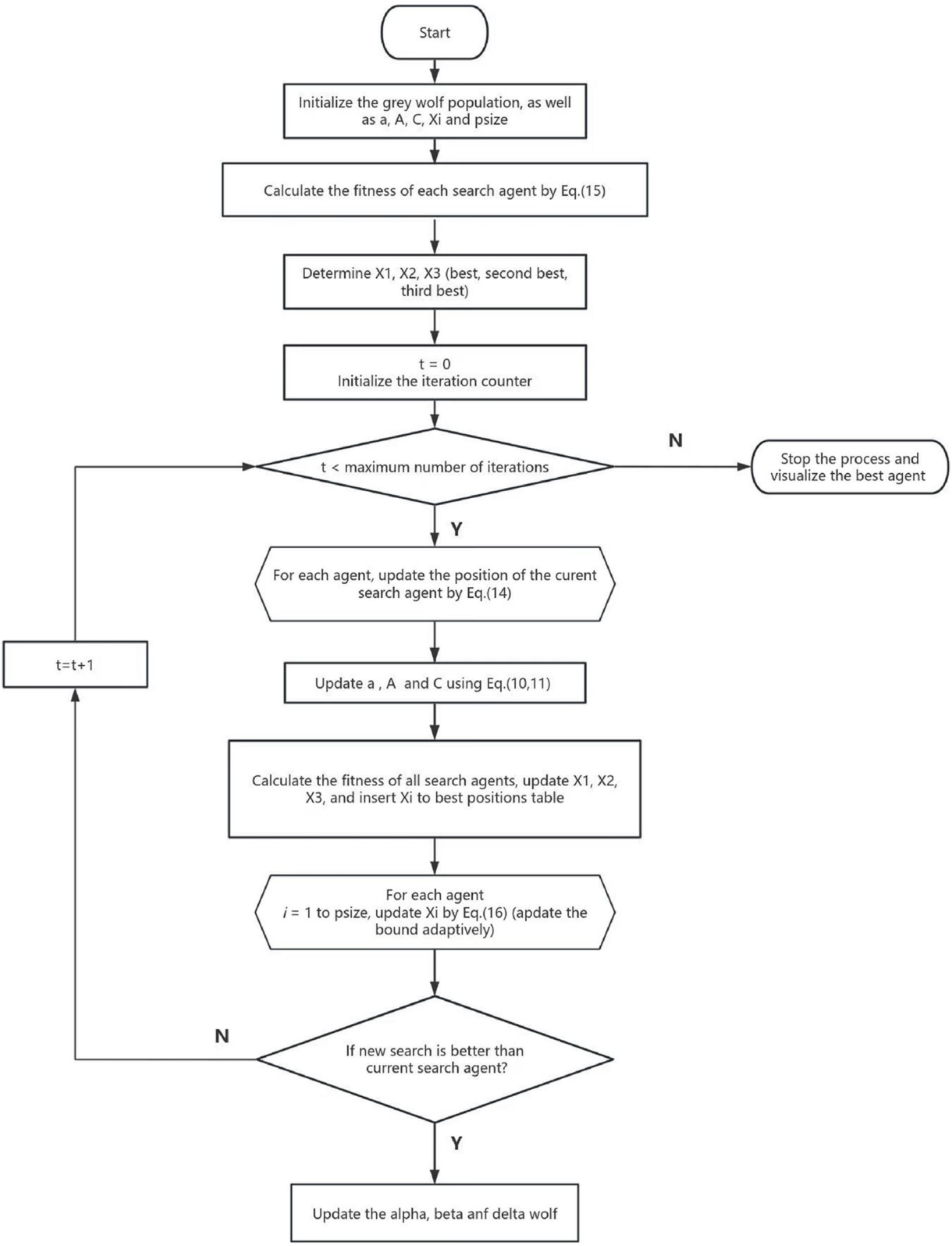


Fig. 5 The flow chart of ABGWO

Table 2 The running parameters values

Parameters	Values
$psize_s$ Small-scale problems population size	100
$psize_l$ Large-scale problems population size	150
e_{bees} The number of employed bees for ABC	16
o_{bees} The number of onlooker bees for ABC	4
c_r Crossover rate for DE	0.9
w_f Weight factor for DE	0.8
p_c Crossover probability for GA	0.95
p_m Mutation probability for GA	0.025
n_{best} The number of best moths to keep for MSA	5
p_f The proportional of first partition for MSA	0.5
c_1 Cognitive factor for PSO	1.2
c_2 Social factor for PSO	1.2
w_{min} Minimum bound on inertia weight for PSO	0.3
w_{max} Maximum bound on inertia weight for PSO	0.9

fewer than m servers, we can consequently reduce the problem's upper bound to y_{best}, setting $m = y_{best}$. This adjustment narrows the search scope of the algorithm. Not only does this expedite convergence, but it also enables the algorithm to explore higher-quality solutions within a smaller search space. This approach enhances the algorithm's efficiency, particularly in scenarios where a vast search space poses challenges for convergence, resulting in suboptimal solutions.

Nevertheless, as $\boldsymbol{X}$ is derived from the previous value of m, it becomes necessary to adjust the individual components of $\boldsymbol{X}$ according to the new boundary when updating the value of m to y_{best}. To accomplish this adjustment, the following equation will be utilized:

$$\begin{aligned} m_{new} &= y_{best} \\ m_{old} &= m \\ k &= \frac{m_{new}}{m_{old}} \\ a_{ij_{new}} &= k \cdot a_{ij}, i \leq p_{size}, j \leq n \end{aligned} \tag{16}$$

Equation (16) completes the reduction of the algorithm's search space for one iteration. Throughout the algorithm's iteration, the search boundary of the ABGWO algorithm undergoes updates based on the current optimal search outcome. The ABGWO will operate according to the pseudocode shown in Algorithm 1, and its flowchart will be presented in Fig. 5.

The time complexity of the ABGWO algorithm is calculated as follows: the time complexity for calculating the fitness value of each individual in the population is $O(N)$, where N is the population size; the time complexity for updating the position of each individual is $O(N^2 + N)$]; the time complexity for the population loop iteration is $O(N^2)$; the time complexity of the optimal point set initialization is negligible. Therefore, the total time complexity of the ABGWO algorithm is $O(N^2)$.

Table 3 The solution quality and stability of various algorithms on small-scale problems (without ABS)

Dim	Algorithms	Best	Worst	Mean	Std
10	ABC	1	3	1.92	0.4
	DE	1	2	1.8	0.4082483
	GA	1	1	1	0
	GWO	1	3	1.08	0.2768875
	MSA	1	1	1	0
	PSO	1	2	1.36	0.4898979
	ABGWO	**1**	**1**	**1**	**0**
20	ABC	4	5	4.36	0.4898979
	DE	3	4	3.12	0.331662479
	GA	1	2	1.92	0.276887462
	GWO	1	2	1.4	0.5
	MSA	1	3	2.04	0.675771164
	PSO	3	5	4.16	0.553774924
	ABGWO	**1**	**2**	**1.36**	**0.4898979**
30	ABC	6	8	7.16	0.746100976
	DE	2	4	3.2	0.645497224
	GA	2	3	2.36	0.489897949
	GWO	1	4	1.68	0.690410506
	MSA	3	7	4.68	1.144552314
	PSO	6	9	7.32	0.748331477
	ABGWO	**1**	**2**	**1.52**	**0.509901951**
40	ABC	8	13	10.36	1.186029792
	DE	2	4	2.68	0.556776436
	GA	4	4	4	0
	GWO	1	2	1.48	0.509901951
	MSA	4	10	7.08	1.15181017
	PSO	9	12	10.72	0.791622806
	ABGWO	**1**	**2**	**1.4**	**0.5**
50	ABC	11	16	13.52	1.194431524
	DE	2	3	2.4	0.5
	GA	5	6	5.64	0.489897949
	GWO	2	2	2	0
	MSA	7	13	9.68	1.725301906
	PSO	13	16	14.48	0.962635272
	ABGWO	**2**	**2**	**2**	**0**

Table 4 The solution quality and stability of various algorithms on small-scale problems (with ABS)

Dim	Algorithms	Best	Worst	Mean	Std
10	ABC*	1	1	1	0
	DE*	1	1	1	0
	GA*	1	1	1	0
	MSA*	1	1	1	0
	PSO*	1	1	1	0
	ABGWO	**1**	**1**	**1**	**0**
20	ABC*	1	2	1.6	0.5
	DE*	1	2	1.32	0.476095229
	GA*	1	2	1.64	0.489897949
	MSA*	1	1	1	0
	PSO*	1	2	1.68	0.476095229
	ABGWO	**1**	**2**	**1.36**	**0.4898979**
30	ABC*	6	8	7.16	0.746100976
	DE*	2	4	3.2	0.645497224
	GA*	2	3	2.36	0.489897949
	MSA*	3	7	4.68	1.144552314
	PSO*	6	9	7.32	0.748331477
	ABGWO	**1**	**2**	**1.52**	**0.509901951**
40	ABC*	8	13	10.36	1.186029792
	DE*	2	4	2.68	0.556776436
	GA*	4	4	4	0
	MSA*	4	10	7.08	1.15181017
	PSO*	9	12	10.72	0.791622806
	ABGWO	**1**	**2**	**1.4**	**0.5**
50	ABC*	11	16	13.52	1.194431524
	DE*	2	3	2.4	0.5
	GA*	5	6	5.64	0.489897949
	MSA*	7	13	9.68	1.725301906
	PSO*	13	16	14.48	0.962635272
	ABGWO	**2**	**2**	**2**	**0**

Algorithm 1 Adaptive boundary grey wolf optimization algorithm

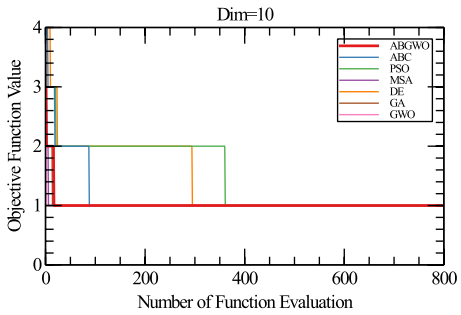
p_{size} represents the number of individuals in population X
$X_i(i = 1, 2, ..., m)$
Initialize a, A and C
Calculate the fitness of each search agent by Eq.(15)
X_1 = the best (or dominating) search agent
X_2 = the second best search agent
X_3 = the third best search agent
while $t <$ maximum number of iterations **do**
  for each search agent **do**
    update the position of the curent search agent by Eq.(14)
  end for
  update a
  update A and C by Eq.(10,11)
  calculate the fitness of all search agents
  update X_1 , X_2 and X_3
  insert X_i to best positions table
  for $i = 1$ to p_{size} **do** //update the bound adaptively
    update X_i by Eq.(16)
  end for
  t = t + 1
end while

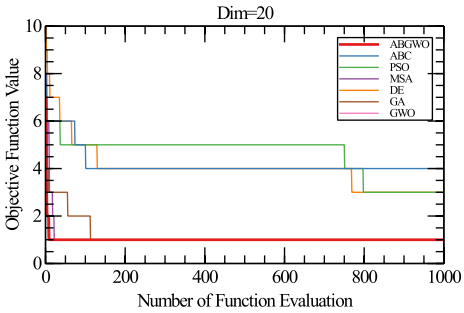
Experimental evaluation

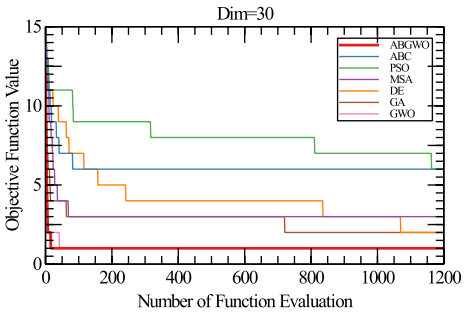
Our experiments utilize Microsoft's publicly available Azure VM packing trace [36], specifically designed for evaluating VM deployment algorithms. We extract VM creation requests, each containing three metrics: VM CPU, memory, and disk. We select the top Dim VM creation requests as input for the algorithm. To thoroughly assess the superior performance of the ABGWO algorithm, we conduct experiments across two different scales: small-scale and large-scale problems. We compare the ABGWO algorithm with GA (genetic algorithm), PSO (particle swarm optimization), MSA (moth search algorithm), ABC (artificial bee colony), DE (differential evolution), and the original GWO algorithm. Additionally, to assess the universality of the proposed adaptive boundary strategy (ABS), we apply it to each of the aforementioned comparison algorithms (the * sign denotes the adaptive boundary version of the algorithm). Furthermore, we verify that the ABS significantly enhances the performance of the GWO algorithm, while its impact on other algorithms is limited. Our experiments are conducted using Python 3.9 on a laptop equipped with an i5

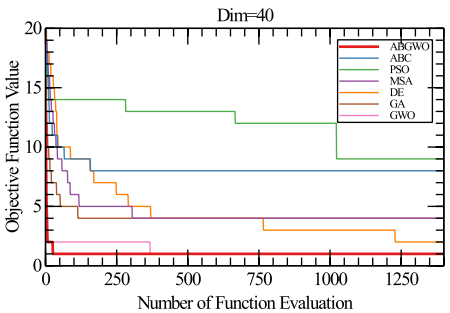
Table 5 Small-scale problems time cost (seconds)

Dim	ABC	DE	GA	MSA	PSO	GWO	ABC*	DE*	GA*	MSA*	PSO*	ABGWO
10	15.2	24.41	29.37	9.53	27.38	5.83	15.13	24.4	29.27	9.46	27.09	**6.24**
20	26.48	29.04	34.58	14.05	33.78	16.11	26.45	28.98	34.55	13.86	32.9	**29.69**
30	31.21	49.24	59.16	22.5	40.8	17.29	31.1	49.2	59.23	22.34	40.14	**79.23**
40	48.33	57.76	67.42	25.94	52.8	32.45	48.32	57.79	67.45	24.77	51.59	**78.24**
50	72.38	92.67	85.77	30.92	73.6	23.41	72.37	92.72	85.8	29.98	72.9	**87.41**









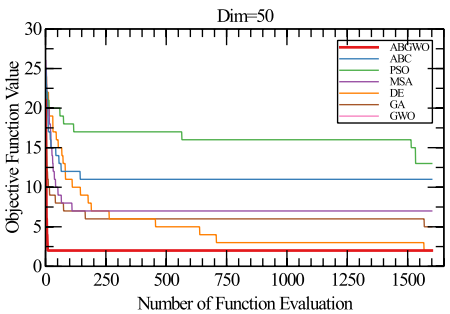
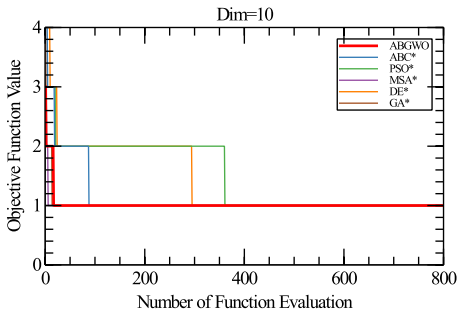
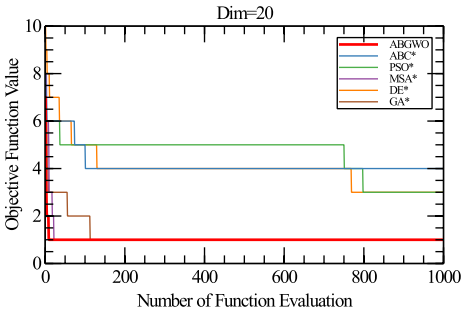
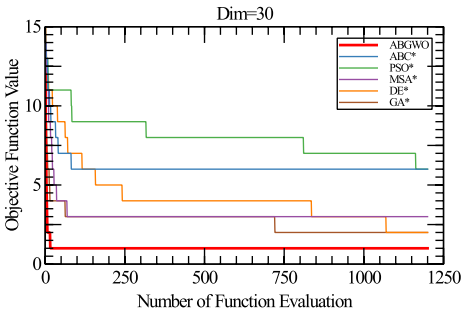
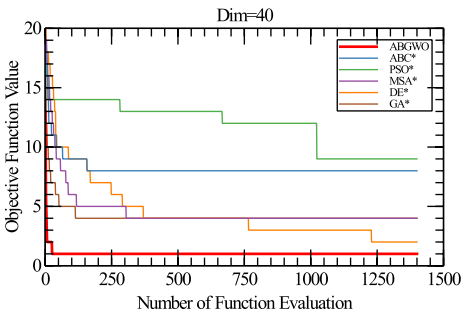


Fig. 6 The small-scale problems convergence result (without ABS)









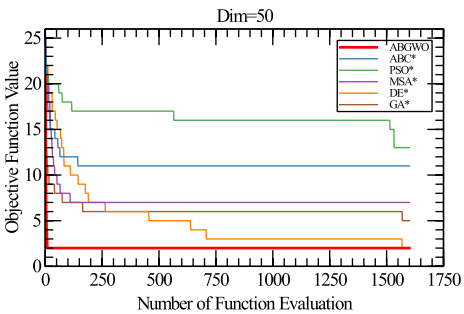


Fig. 7 The small-scale problems convergence result (with ABS)

Table 6 The solution quality and stability of various algorithms on large-scale problems (without ABS)

Dim	Algorithms	Best	Worst	Mean	Std
1000	ABC	484	501	493.6	5.803939549
	DE	575	592	585.3333333	5.2734736
	GA	275	306	292.0666667	10.45033606
	GWO	249	319	270.4	18.18869351
	MSA	316	388	357.6	21.92454593
	PSO	393	434	416.2	10.35926087
	ABGWO	**153**	**254**	**210.25**	**37.39607823**
1500	ABC	774	793	782.1333333	5.527420823
	DE	890	911	900.4	5.40898723
	GA	414	469	432.3333333	14.71474803
	GWO	335	415	368.7333333	20.55468896
	MSA	531	599	562.4666667	16.2078758
	PSO	593	626	608.0666667	9.42994218
	ABGWO	**178**	**354**	**297.55**	**54.63898261**
2000	ABC	1068	1085	1076.666667	5.313952883
	DE	1199	1224	1215.6	7.60450994
	GA	540	590	568.2	13.62875112
	GWO	398	502	462.7333333	33.57606857
	MSA	724	911	788.7333333	52.3865668
	PSO	779	838	814.8	14.04177441
	ABGWO	**318**	**455**	**420.5625**	**45.12939729**
2500	ABC	1381	1415	1394.666667	8.103497187
	DE	1520	1550	1536.733333	8.413141648
	GA	682	1204	781.4	122.2718283
	GWO	501	838	660.0666667	91.00193614
	MSA	925	1096	996.1333333	43.08275536
	PSO	1034	1098	1066.466667	16.72409155
	ABGWO	**232**	**392**	**296.25**	**51.17727656**
3000	ABC	1686	1711	1701.266667	8.867812314
	DE	1828	1866	1850.066667	10.61983768
	GA	831	1320	1019.8	159.8808485
	GWO	660	999	822.5333333	95.88748367
	MSA	1113	1310	1182.266667	46.22841839
	PSO	1225	1339	1275.666667	33.02091689
	ABGWO	**296**	**473**	**385.1818182**	**49.56171543**

8250U processor and 8GB of RAM. The running parameters for each algorithm are presented in Table 2.

Small-scale problems

In this section, we address 5 low-dimensional problems with Dim ranging from 10 to 50. The comparison algorithm mentioned above is applied to these 5 problems, with iterations set at 800, 1000, 1200, 1400, and 1600 for each problem dimension. Each algorithm is independently executed 25 times for each dimension of the problem.

Table 7 The solution quality and stability of various algorithms on large-scale problems (with ABS)

Dim	Algorithms	Best	Worst	Mean	Std
1000	ABC*	189	215	208.7333333	7.176018262
	DE*	252	325	283.8666667	23.48515235
	GA*	345	560	413.6	68.50005214
	MSA*	264	350	317.3333333	27.42956347
	PSO*	215	230	222.6666667	4.151878519
	ABGWO	**153**	**254**	**210.25**	**37.39607823**
1500	ABC*	286	309	300.7333333	6.111659427
	DE*	890	911	900.4	5.40898723
	GA*	438	619	529.5333333	53.17607769
	MSA*	336	529	460	55.62373594
	PSO*	304	326	315.2	6.049793385
	ABGWO	**178**	**354**	**297.55**	**54.63898261**
2000	ABC*	373	386	380.9333333	4.06143301
	DE*	535	618	579	30.76872252
	GA*	555	803	707.7333333	71.92006144
	MSA*	488	687	583.2	62.69677595
	PSO*	382	406	396.8	6.493953231
	ABGWO	**318**	**455**	**420.5625**	**45.12939729**
2500	ABC*	602	670	631.2666667	21.0423609
	DE*	850	1215	1067.066667	116.1830247
	GA*	967	1542	1342.933333	183.9598041
	MSA*	596	1214	989.6	143.979562
	PSO*	644	703	675.0666667	16.2896403
	ABGWO	**232**	**392**	**296.25**	**51.17727656**
3000	ABC*	726	832	769.6	23.59116057
	DE*	1223	1887	1613.891284	231.8237281
	GA*	1017	1772	1602.866667	248.429601
	MSA*	836	1467	1177.933333	217.6656895
	PSO*	756	858	801.4	27.4689435
	ABGWO	**296**	**473**	**385.1818182**	**49.56171543**

Table 3 presents the statistical outcomes of ABGWO and comparison algorithms with ABS disabled. It is evident that as the problem size increases, most algorithms without ABS exhibit deteriorating solution quality and solution stability compared to the ABGWO algorithm. Only the DE algorithm maintains satisfactory performance, while both the ABC and PSO algorithms experience a sharp decline in both optimal solution quality and solution stability. The GWO algorithm is equivalent to the ABGWO algorithm in terms of optimal solution quality and stability. Meanwhile, the solution quality of the ABGWO algorithm consistently outperforms others, demonstrating steady results.

Table 4 illustrates the operational outcomes of each algorithm under the ABS mechanism. For comparative analysis of ABGWO's performance, both Tables 3 and 4 present the statistical findings of the ABGWO algorithm.

It is evident that the statistical outcomes of ABGWO in both tables remain consistent. Clearly, ABS notably enhances the solution quality and stability of each algorithm, especially noticeable in the case of the MSA algorithm

Figure 6 depicts the convergence plots of each algorithm excluding ABS. It is evident that ABGWO achieves rapid convergence across these five small-scale problems and consistently converges to the highest-quality solution. Conversely, ABC , DE and PSO exhibit slower convergence with a consistent number of iterations and yield the lowest-quality optimal solutions. The remaining algorithms demonstrate varying convergence trends for each problem, highlighting their instability.

Table 5 shows the running time statistics of each algorithm. ABS can improve the performance of other algorithms on small-scale problems, although ABGWO has a moderate running time performance on small-scale problems, based on the data in Tables 4 and 5, ABGWO performs the best compared to other algorithms.

Figure 7 illustrates the convergence plots of each algorithm while utilizing the ABS. It vividly demonstrates the significant enhancement in convergence speed achieved by the ABS across various algorithms, particularly evident in resolving small-scale problems and achieving high-quality solutions at convergence. With the ABS's influence, most algorithms exhibit convergence after only a few dozen iterations.

Large-scale problems

In the preceding section, we exclusively examined the performance of ABGWO on small-scale problems. While the addition of ABS to the comparative algorithm can enhance its performance, ABGWO remains superior in terms of overall performance. This section delves into the performance of ABGWO on large-scale VMPPs. Specifically, this subsection addresses five high-dimensional problems with Dim ranging from 1000 to 3000. For each of these five problems, we subject the comparison algorithm to iterations ranging from 3000 to 5000, conducting 30 independent runs for each different dimension of the problem.

Table 6 presents the statistical outcomes of ABGWO and comparative algorithms with ABS enabled. It is evident that the ABGWO algorithm exhibits remarkable superiority in large-scale VMPPs. The ABGWO algorithm consistently delivers the highest-quality solutions across all five large-scale problems. Although the standard deviation of ABGWO increases with problem size, its best solution always outperforms the optimal solution of other algorithms. Conversely, as problem size escalates, the standard deviation of DE and ABC remains low, but their solution quality deteriorates significantly, highlighting the challenge of finding high-quality solutions for DE and ABC as problem size increases. GWO maintains a low standard deviation while ensuring better solution quality, indicating the stability of both GWO and ABGWO. Although GA yields good optimal solutions for certain problems, it exhibits the highest standard deviation, suggesting less stability in handling large-scale problems. The performance of other algorithms does not stand out significantly.

Table 7 presents the performance statistics of each algorithm when utilizing the ABS. As observed in the experiments on small-scale problems, the statistical outcomes of ABGWO in both Tables 5 and 6 correspond to the same version of the ABGWO algorithm incorporating the ABS mechanism. Evidently, ABS contributes to enhancing the solution quality of each algorithm to some degree in large-scale problems. However, the high standard deviation indicates that this enhancement in solution quality is notably unstable, and the gap between the improved optimal solution and the optimal solution of ABGWO is considerable. Overall, while the ABS yields some improvements in the performance of the comparison algorithm on large-scale problems, these improvements are substantially less pronounced compared to those observed in small-scale problems.

Figure 8 illustrates the convergence plots of each algorithm in addressing the large-scale VM deployment problem without the use of ABS. It is evident that both DE and ABC algorithms struggle to converge on these five large-scale problems, with PSO, MSA and GWO demonstrating a smoother convergence rate. However, their final outcomes are not as satisfactory as those achieved

Table 8 Large-scale problems time cost (seconds)

Dim	ABC	DE	GA	MSA	PSO	GWO	ABC*	DE*	GA*	MSA*	PSO*	ABGWO
1000	2509.74	2328.49	2373.08	1122.34	2366.6	723.69	2508.72	2331.72	2378.45	1119.28	2365.78	**777.23**
1500	4237.65	3933.56	3960.37	1963.37	3930.34	1107.15	4238.53	3938.07	3972.9	1958.7	3928.84	**1187.56**
2000	5628.16	5025.43	5114.31	2931.84	5055.07	1467.56	5625.24	5031.11	5115.72	2923.87	5043.47	**1786.09**
2500	7885.27	7963.43	7858.56	3521.35	7849.57	2051.1	7882.67	7970.49	7855.5	3510.82	7826.34	**2490.28**
3000	8694.25	8528.74	8527.15	5666.49	8533.76	3615.17	8687.37	8535.21	8517.4	5658.99	8507.11	**3473.72**

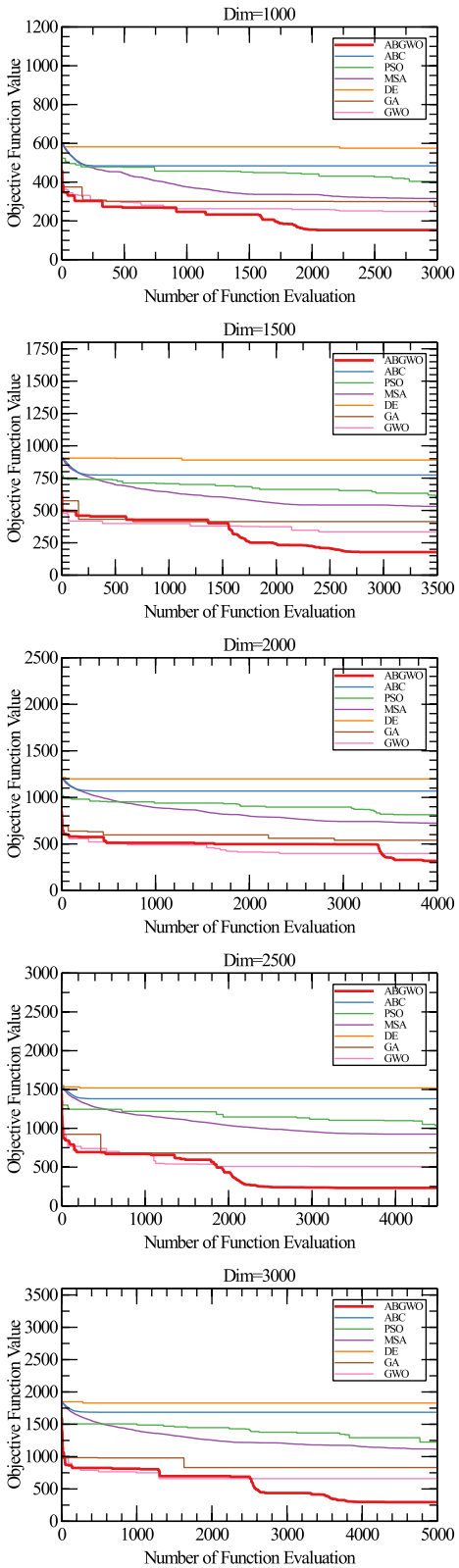


Fig. 8 The large-scale problems convergence result (without ABS)

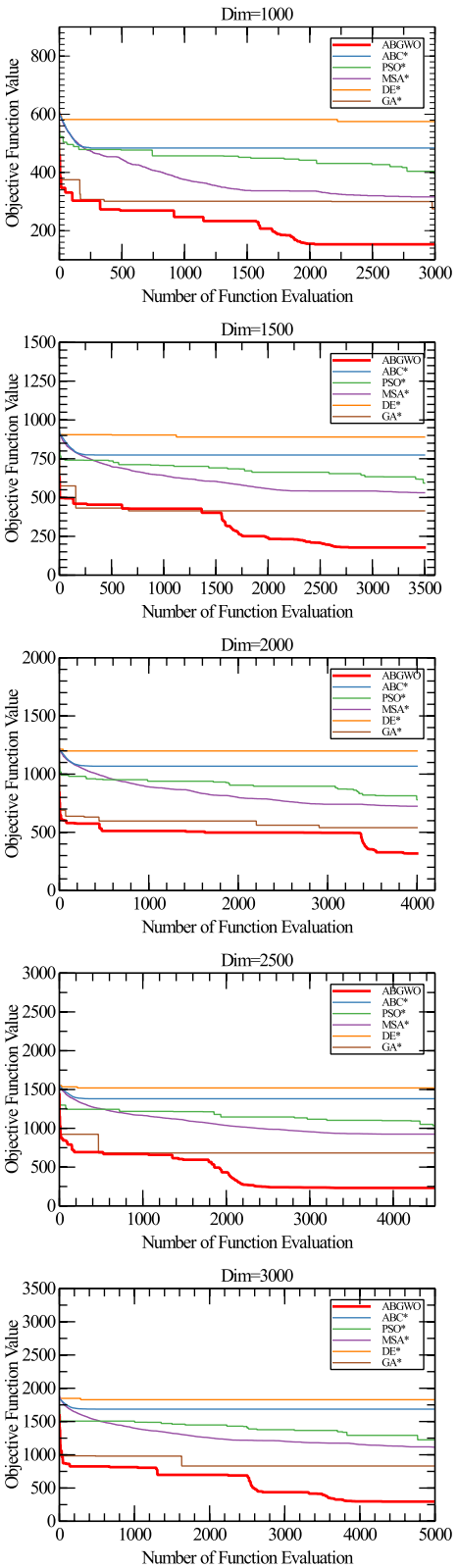


Fig. 9 The large-scale problems convergence result (with ABS)

by the GA algorithm under the uniform iteration limit. GWO exhibits the best convergence speed and results among all the compared algorithms. Nevertheless, there remains a significant gap between GA and the ABGWO algorithm, which excels in terms of running speed, convergence speed, and convergence outcome.

Table 8 shows the running time statistics of each algorithm on the large-scale problem. It can be seen from the figure that as the problem size increases dramatically, the comparison algorithms run slower and slower, and although MSA is much faster compared to the other algorithms, its running time is still almost 1.5 times that of ABGWO, which indicates that ABGWO is already running much faster than the other algorithms.

Figure 9 illustrates the convergence of each algorithm on the large-scale problem using the ABS method, demonstrating the effectiveness of ABS in handling large-scale problems. With ABS's influence, the GA algorithm achieves a convergence solution second only to the ABGWO algorithm, significantly improving its convergence speed. While the ABC algorithm shows a smaller enhancement in convergence optimal solution, its convergence speed is also accelerated. Overall, ABS enhances both convergence speed and the quality of optimal solutions for each comparison algorithm on large-scale problems, although the improvement is not as pronounced as observed in small-scale problems. This is attributed to the increasing gap between the converged optimal solutions of comparison algorithms and the optimal solutions of the ABGWO algorithm as problem size increases.

The experiments show that adaptive boundary strategy can greatly improve the convergence speed and solution quality of the comparison algorithm on small-scale problems, while the improvement is smaller on large-scale problems, which reflects the universality of the adaptive boundary strategy. The ABGWO algorithm achieves excellent results on both small-scale and large-scale problems. The ABGWO algorithm outperforms or equals other algorithms on small-scale problems, while ABGWO completely outperforms other algorithms on large-scale problems, and the running speed, convergence, and robustness of the ABGWO algorithm are significantly better than other comparative algorithms.

Conclusion

In this paper, we introduce a novel approach, termed the Adaptive Boundary Grey Wolf Optimization Algorithm (ABGWO), to address the virtual machine deployment issue in cloud computing. Leveraging the ABS, the ABGWO algorithm dynamically adjusts the search boundaries, effectively reducing the search space.The proposed ABS and ABGWO algorithm demonstrate versatility across a wide range of multidimensional vector bin packing problems with decimal encoding. Experimental results reveal that the ABS significantly enhances convergence speed and solution quality for small-scale problems. While the impact is somewhat diminished for larger-scale problems, the universal applicability of the ABS remains evident.

Notably, the ABGWO algorithm delivers exceptional performance across both small-scale and large-scale problems. It either matches or surpasses other algorithms for small-scale problems and outperforms them comprehensively for larger-scale problems. Moreover, ABGWO exhibits superior optimal solution quality compared to its counterparts.With the proliferation of communication-intensive applications such as MapReduce [37], the data center network's bandwidth limitations are increasingly becoming a bottleneck for application performance and scalability. Therefore, in future research, we plan to speed up algorithm convergence, reduce running time, and improve the stability of the algorithm, while exploring solutions to address the constraints of data center network bandwidth to optimize virtual machine placement.

Authors' contributions

Haoyu Li and Hao Feng completed the main manuscript text and figures; Haoyu Li, Hao Feng and Kun Cao performed the experiment; Xiumin Zhou and Yuming Liu analyzed the experimental results and prepare the result figures. All authors reviewed the manuscript.

Data availability

The dataset used in this paper is from Microsoft's public dataset Azure VM packing trace, https://github.com/Azure/AzurePublicDataset. All of the material is owned by the authors and/or no permissions are required.

Declarations

Competing interests

The authors declare no competing interests.

Received: 14 June 2024 Accepted: 21 January 2025
Published online: 13 February 2025

References

1. Katal A, Dahiya S, Choudhury T (2023) Energy efficiency in cloud computing data centers: a survey on software technologies. Clust Comput 26(3):1845–1875
2. Hsieh SY, Liu CS, Buyya R, Zomaya AY (2020) Utilization-prediction-aware virtual machine consolidation approach for energy-efficient cloud data centers. J Parallel Distrib Comput 139:99–109
3. Patros P, Spillner J, Papadopoulos AV, Varghese B, Rana O, Dustdar S (2021) Toward sustainable serverless computing. IEEE Internet Comput 25(6):42–50
4. Masdari M, Gharehpasha S, Ghobaei-Arani M, Ghasemi V (2020) Bio-inspired virtual machine placement schemes in cloud computing environment: taxonomy, review, and future research directions. Clust Comput 23(4):2533–2563

5. Gharehpasha S, Masdari M, Jafarian A (2021) Power efficient virtual machine placement in cloud data centers with a discrete and chaotic hybrid optimization algorithm. Clust Comput 24(2):1293–1315
6. Singh J, Bajaj R, Kumar A (2021) Scaling down power utilization with optimal virtual machine placement scheme for cloud data center resources: A performance evaluation. In: 2021 2nd Global Conference for Advancement in Technology (GCAT), IEEE, pp 1–6
7. Talebian H, Gani A, Sookhak M, Abdelatif AA, Yousafzai A, Vasilakos AV, Yu FR (2020) Optimizing virtual machine placement in iaas data centers: taxonomy, review and open issues. Clust Comput 23:837–878
8. Gharehpasha S, Masdari M, Jafarian A (2021) Virtual machine placement in cloud data centers using a hybrid multi-verse optimization algorithm. Artif Intell Rev 54(3):2221–2257
9. Abohamama AS, Hamouda E (2020) A hybrid energy-aware virtual machine placement algorithm for cloud environments. Expert Syst Appl 150:113306
10. Sandeep S (2022) Almost optimal inapproximability of multidimensional packing problems. In: 2021 IEEE 62nd Annual Symposium on Foundations of Computer Science (FOCS), IEEE, pp 245–256
11. Laabadi S (2020) Metaheuristic approaches for solving packing problems: 0/1 multidimensional knapsack problem and two-dimensional bin packing problem as examples. PhD thesis, Université Hassan II Casablanca, Maroc
12. Pandey A (2023) An analysis of solutions to the 2d bin packing problem and additional complexities. Authorea Preprints
13. Mao Y, Mitra A, Sundaram S, Tabuada P (2022) On the computational complexity of the secure state-reconstruction problem. Automatica 136:110083
14. Gill SS, Tuli S, Xu M, Singh I, Singh KV, Lindsay D, Tuli S, Smirnova D, Singh M, Jain U et al (2019) Transformative effects of iot, blockchain and artificial intelligence on cloud computing: Evolution, vision, trends and open challenges. Internet Things 8:100118
15. Mirjalili S, Mirjalili SM, Lewis A (2014) Grey wolf optimizer. Adv Eng Softw 69:46–61
16. Makhadmeh S N, Al-Betar M A, Doush I A, et al. Recent advances in Grey Wolf Optimizer, its versions and applications[J]. IEEE Access, 2023.
17. Faris H, Aljarah I, Al-Betar MA, Mirjalili S (2018) Grey wolf optimizer: a review of recent variants and applications. Neural Comput & Applic 30:413–435
18. Jyoti A, Shrimali M (2020) Dynamic provisioning of resources based on load balancing and service broker policy in cloud computing. Clust Comput 23(1):377–395
19. Belgacem A, Beghdad-Bey K, Nacer H, Bouznad S (2020) Efficient dynamic resource allocation method for cloud computing environment. Clust Comput 23(4):2871–2889
20. Lavanya M, Shanthi B, Saravanan S (2020) Multi objective task scheduling algorithm based on sla and processing time suitable for cloud environment. Comput Commun 151:183–195
21. Liu B, Li J, Lin W, Bai W, Li P, Gao Q (2021) K-pso: An improved pso-based container scheduling algorithm for big data applications. Int J Netw Manag 31(2):e2092
22. Prasanna Kumar K, Kousalya K (2020) Amelioration of task scheduling in cloud computing using crow search algorithm. Neural Comput & Applic 32(10):5901–5907
23. Kumar M, Sharma SC, Goel S, Mishra SK, Husain A (2020) Autonomic cloud resource provisioning and scheduling using meta-heuristic algorithm. Neural Comput & Applic 32:18285–18303
24. Ghobaei-Arani M, Rahmanian AA, Shamsi M, Rasouli-Kenari A (2018) A learning-based approach for virtual machine placement in cloud data centers. Int J Commun Syst 31(8):e3537
25. Wang H, Tianfield H (2018) Energy-aware dynamic virtual machine consolidation for cloud datacenters. IEEE Access 6:15259–15273
26. Rashida SY, Sabaei M, Ebadzadeh MM, Rahmani AM (2020) A memetic grouping genetic algorithm for cost efficient vm placement in multi-cloud environment. Clust Comput 23:797–836
27. Azizi S, Shojafar M, Abawajy J, Buyya R (2020) Grvmp: a greedy randomized algorithm for virtual machine placement in cloud data centers. IEEE Syst J 15(2):2571–2582
28. Duong-Ba T, Tran T, Nguyen T, Bose B (2018) A dynamic virtual machine placement and migration scheme for data centers. IEEE Trans Serv Comput 14(2):329–341
29. Tabrizchi H, Kuchaki Rafsanjani M (2021) Energy refining balance with ant colony system for cloud placement machines. J Grid Comput 19(1):7
30. Baalamurugan K, Vijay Bhanu S (2020) A multi-objective krill herd algorithm for virtual machine placement in cloud computing. J Supercomput 76(6):4525–4542
31. Caviglione L, Gaggero M, Paolucci M, Ronco R (2021) Deep reinforcement learning for multi-objective placement of virtual machines in cloud datacenters. Soft Comput 25(19):12569–12588
32. Azizi S, Zandsalimi M, Li D (2020) An energy-efficient algorithm for virtual machine placement optimization in cloud data centers. Clust Comput 23(4):3421–3434
33. Tseng FH, Jheng YM, Chou LD, Chao HC, Leung VC (2017) Link-aware virtual machine placement for cloud services based on service-oriented architecture. IEEE Trans Cloud Comput 8(4):989–1002
34. Tripathi A, Pathak I, Vidyarthi DP (2020) Modified dragonfly algorithm for optimal virtual machine placement in cloud computing. J Netw Syst Manag 28:1316–1342
35. Bahreininejad A (2019) Improving the performance of water cycle algorithm using augmented lagrangian method. Adv Eng Softw 132:55–64
36. (2019) Microsoft: Azure vm packing trace. https://github.com/Azure/AzurePublicDataset/. Accessed 1 June 2024
37. Dean J, Ghemawat S (2008) MapReduce: simplified data processing on large clusters. Commun ACM 51(1):107–113

Publisher's Note